

BI Vendas — Painel Comercial

Análise Descritiva de Vendas Comerciais

Backend: Flask + Pandas · Frontend: HTML + Chart.js

Linguagem	Python 3.x + JavaScript
Framework Backend	Flask + Flask-CORS
Biblioteca de Dados	Pandas + OpenPyXL
Visualização	Chart.js 4.4
Arquivo de Dados	dados_comerciais.xlsx

1. Estrutura do Projeto

O sistema é composto por duas partes independentes que se comunicam via HTTP. O backend processa os dados do arquivo Excel e expõe endpoints REST. O frontend consome esses endpoints e renderiza os gráficos e a tabela de ranking.

Arquivo	Responsabilidade
backend/app.py	API Flask com todos os endpoints de análise
backend/dados_comerciais.xlsx	Base de dados comerciais (fonte única)
frontend/index.html	Painel web com gráficos e tabela interativa

2. Backend — app.py (Flask)

2.1 Dependências e configuração inicial

O backend utiliza Flask como servidor HTTP, Pandas para processamento dos dados do Excel e Flask-CORS para permitir que o frontend (rodando como arquivo local) consiga fazer requisições à API sem bloqueio de CORS.

```
import os
import pandas as pd
from flask import Flask, jsonify
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

DATA_PATH = os.path.join(os.path.dirname(__file__), "dados_comerciais.xlsx")

def carregar_dados():
    df = pd.read_excel(DATA_PATH)
    df.columns = df.columns.str.strip()
    df["Quantidade"] = pd.to_numeric(df["Quantidade"], errors="coerce").fillna(0)
    df["Receita"] = pd.to_numeric(df["Receita"], errors="coerce").fillna(0)
    return df
```

2.2 Endpoint /api/resumo — KPIs Gerais

Retorna os indicadores-chave do negócio: receita total, volume de itens vendidos, quantidade de pedidos e ticket médio por pedido.

```
@app.route("/api/resumo", methods=["GET"])
def resumo():
    df = carregar_dados()
    total_receita = round(df["Receita"].sum(), 2)
    total_itens = int(df["Quantidade"].sum())
    total_pedidos = len(df)
    ticket_medio = round(total_receita / total_pedidos, 2) if total_pedidos > 0 else 0

    return jsonify({
        "total_receita": total_receita,
        "total_itens": total_itens,
        "total_pedidos": total_pedidos,
        "ticket_medio": ticket_medio
    })
```

2.3 Endpoint /api/por-produto — Agregação por Produto

Agrupa os dados pela coluna Produto usando groupby do Pandas, calculando a soma de Quantidade e Receita. O resultado é ordenado por receita decrescente.

```
@app.route("/api/por-produto", methods=["GET"])
def por_produto():
```

```

df = carregar_dados()
agrupado = (
    df.groupby("Produto")
    .agg(quantidade=("Quantidade", "sum"), receita=("Receita", "sum"))
    .reset_index()
    .sort_values("receita", ascending=False)
)
agrupado["receita"] = agrupado["receita"].round(2)
agrupado["quantidade"] = agrupado["quantidade"].astype(int)
return jsonify(agrupado.to_dict(orient="records"))

```

2.4 Endpoint /api/por-localidade — Agregação por Localidade

```

@app.route("/api/por-localidade", methods=["GET"])
def por_localidade():
    df = carregar_dados()
    agrupado = (
        df.groupby("Localidade")
        .agg(quantidade=("Quantidade", "sum"), receita=("Receita", "sum"))
        .reset_index()
        .sort_values("receita", ascending=False)
    )
    agrupado["receita"] = agrupado["receita"].round(2)
    agrupado["quantidade"] = agrupado["quantidade"].astype(int)
    return jsonify(agrupado.to_dict(orient="records"))

```

2.5 Endpoint /api/por-mes — Evolução Mensal

```

@app.route("/api/por-mes", methods=["GET"])
def por_mes():
    df = carregar_dados()
    df["Data"] = pd.to_datetime(df["Data"], errors="coerce")
    df["Mes"] = df["Data"].dt.to_period("M").astype(str)
    agrupado = (
        df.groupby("Mes")
        .agg(quantidade=("Quantidade", "sum"), receita=("Receita", "sum"))
        .reset_index()
        .sort_values("Mes")
    )
    agrupado["receita"] = agrupado["receita"].round(2)
    agrupado["quantidade"] = agrupado["quantidade"].astype(int)
    return jsonify(agrupado.to_dict(orient="records"))

if __name__ == "__main__":
    app.run(debug=True, port=5000)

```

3. Frontend — index.html

3.1 Estrutura da página e estilos

A página é um arquivo HTML único, sem dependências externas além do Chart.js (carregado via CDN). O layout usa CSS Grid responsivo: cards de KPI no topo, dois gráficos lado a lado no centro e tabela de ranking na base. A troca entre visão por produto e por localidade é feita via abas, sem recarregar a página.

3.2 Carregamento de KPIs

```
async function carregarResumo() {
  try {
    const res = await fetch(`${BASE}/resumo`);
    const data = await res.json();

    document.getElementById('kpi-receita').textContent = formatBRL(data.total_receita);
    document.getElementById('kpi-itens').textContent = formatNum(data.total_itens);
    document.getElementById('kpi-pedidos').textContent = formatNum(data.total_pedidos);
    document.getElementById('kpi-ticket').textContent = formatBRL(data.ticket_medio);
  } catch (e) {
    console.error('Erro ao carregar resumo:', e);
  }
}
```

3.3 Gráfico de Colunas (Chart.js — tipo bar)

Recebe os labels (nomes dos produtos ou localidades) e os valores de receita. Cada barra tem cor distinta, bordas arredondadas e tooltip formatado em BRL.

```
function renderizarBarras(container, labels, valores, label) {
  container.innerHTML = '';
  const ctx = document.getElementById('canvas-barras').getContext('2d');

  chartBarras = new Chart(ctx, {
    type: 'bar',
    data: {
      labels,
      datasets: [{
        label,
        data: valores,
        backgroundColor: CORES,
        borderRadius: 6,
        borderSkipped: false
      }]
    },
    options: {
      responsive: true,
      maintainAspectRatio: false,
      plugins: {
        legend: { display: false },
        tooltip: {
```

```

        callbacks: { label: ctx => ' ' + formatBRL(ctx.raw) }
      }
    },
    scales: {
      x: { grid: { display: false } },
      y: { ticks: { callback: v => 'R$ ' + (v/1000).toFixed(0) + 'k' } }
    }
  }
});
}

```

3.4 Gráfico de Pizza (Chart.js — tipo doughnut)

Exibe a distribuição percentual de quantidade vendida. O tooltip mostra o valor absoluto e o percentual calculado no lado cliente. A legenda fica posicionada abaixo do gráfico.

```

function renderizarPizza(container, labels, valores) {
  container.innerHTML = '';
  const ctx = document.getElementById('canvas-pizza').getContext('2d');

  chartPizza = new Chart(ctx, {
    type: 'doughnut',
    data: {
      labels,
      datasets: [{
        data: valores,
        backgroundColor: CORES,
        borderWidth: 2,
        borderColor: '#fff',
        hoverOffset: 8
      }]
    },
    options: {
      cutout: '55%',
      plugins: {
        legend: { position: 'bottom' },
        tooltip: {
          callbacks: {
            label: ctx => {
              const total = ctx.dataset.data.reduce((a, b) => a + b, 0);
              return ` ${ctx.label}: ${formatNum(ctx.raw)} (${pct(ctx.raw, total)})`;
            }
          }
        }
      }
    }
  });
}

```

4. Documentação dos Endpoints da API

GET /api/status

Verifica se a API está online.

Resposta:

```
{ "status": "online", "versao": "1.0.0" }
```

GET /api/resumo

Retorna KPIs gerais: receita total, itens vendidos, pedidos e ticket médio.

Resposta:

```
{ "total_receita": 1254830.50, "total_itens": 3921, "total_pedidos": 500, "ticket_medio": 2509.66 }
```

GET /api/por-produto

Retorna lista de produtos com quantidade e receita agregadas, ordenado por receita.

Resposta:

```
[ { "Produto": "Notebook Pro", "quantidade": 480, "receita": 412500.00 }, ... ]
```

GET /api/por-localidade

Retorna lista de localidades com quantidade e receita agregadas, ordenado por receita.

Resposta:

```
[ { "Localidade": "Sao Paulo", "quantidade": 620, "receita": 310000.00 }, ... ]
```

GET /api/por-mes

Retorna volumetria e receita por mes no formato YYYY-MM, em ordem cronologica.

Resposta:

```
[ { "Mes": "2024-01", "quantidade": 310, "receita": 98000.00 }, ... ]
```

5. Como Executar o Sistema

Passo 1	Instalar dependências Python <code>pip install flask pandas openpyxl flask-cors</code>
Passo 2	Colocar o arquivo Excel na pasta backend/ O arquivo deve ter as colunas: Data, Produto, Localidade, Quantidade, Receita
Passo 3	Iniciar o servidor Flask <code>cd backend python app.py</code>
Passo 4	Abrir o frontend no navegador Abra o arquivo <code>frontend/index.html</code> diretamente no navegador

Após iniciar o backend, a API estará disponível em **http://localhost:5000**. O painel carrega automaticamente os dados ao abrir o arquivo HTML no navegador.